

FINAL EXAM - 2025

ADVANCED DATABASE

Kayla El-Raudhia De Nato	1232001059
Kayla Mahavira	1232001053
Najwa Naela Fawwaz	1232001010

OUTLINE



Contribution & Rating Teammates



Background



Purpose of Solving



Method & Architecture Staging Diagram



Analysis & Results Query



Conclusion, References, Acknowledge

CONTRIBUTION

Kayla El-Raudhia De Nato

- Database Semi-Structured Data (XML)
- Academic Report
- Video Documentation (Portfolio)

Najwa Naela Fawwaz

- NoSQL - DocumentDB
- Academic Report
- Video Documentation (Portfolio)

Kayla Mahavira

- NoSQL - GraphDB
- Academic Report
- Video Documentation (Portfolio)

RATING TEAMMATES

Nama Rekan	NIM	Skor	Narasi Penilaian
Kayla El-Raudhia De Nato	1232001059	4	Sangat bagus. Kayla menunjukkan pemahaman mendalam terkait XML dan XPath/XQuery. Ia berhasil menerjemahkan struktur RDBMS menjadi model XML tree dengan baik, menyusun query XPath dan XQuery secara lengkap, dan melakukan analisis hasil dengan rapi. Validasi XML berhasil dilakukan, serta file struktur, query, dan screenshot disusun sesuai instruksi dosen. Kontribusinya sangat dominan di bagian semi-structured data.
Najwa Naela Fawwaz	1232001010	4	Sangat Bagus. Najwa menyelesaikan bagian DocumentDB dengan baik, termasuk proses konversi data dari XML ke JSON, eksekusi query MongoDB, serta visualisasi chart yang relevan. Ia juga berhasil mengatur Data Federation agar dosen dapat mengakses database. Meskipun terdapat sedikit kendala saat pengumpulan file, seluruh instruksi tugas tetap terpenuhi.
Kayla Mahavira	1232001053	4	Sangat bagus. Kayla Mahavira berhasil mengubah data JSON menjadi graph model di Neo4j dengan struktur node dan relasi yang rapi. Ia menunjukkan pemahaman kuat terhadap Cypher query, termasuk eksplorasi pola menggunakan shortest-path dan variable length pattern. Visualisasi data dan relasi juga sangat informatif. Dokumentasi lengkap dan tepat waktu.

BACKGROUND

Background of the problem

RBL Bank di India dan Kementerian Dalam Negeri Republik Indonesia (Kemendagri) berhasil melakukan transformasi digital dengan memanfaatkan Database Management System (DBMS) dan platform cloud dari Nutanix. RBL Bank yang sebelumnya menghadapi lambatnya login sistem, biaya penyimpanan tinggi, dan keterbatasan manajemen database, mengimplementasikan Nutanix Enterprise Cloud untuk mempercepat provisioning database hingga 90%, mengurangi jejak hardware 40%, dan meningkatkan efisiensi operasional. Sementara itu, Kemendagri menghadapi tantangan integrasi data antar instansi dan keterlambatan akses data, sehingga meluncurkan Sistem Informasi Pembangunan Daerah (SIPD) sebagai bagian dari inisiatif Satu Data Indonesia untuk memperkuat kolaborasi, meningkatkan keterbukaan data, dan mendorong transparansi keuangan daerah.

Transformasi ini menghasilkan dampak signifikan bagi kedua institusi, di mana RBL Bank mampu mengurangi penyimpanan hingga 90TB dan mempercepat kinerja, sedangkan Kemendagri berhasil meningkatkan keakuratan data dan mempercepat proses perencanaan pembangunan. Solusi Nutanix memberikan keunggulan berupa manajemen database terpusat, perlindungan data, integrasi multi-cloud, dan kontrol akses berbasis jejak digital yang mendorong tata kelola data yang lebih efisien dan responsif bagi sektor perbankan dan pemerintahan.

RDBMS/OODB TO SEMI-STRUCTURED SOLUTIONS DOCDB (MONGODB)

RDBMS to Semi-Structured: DocDB

Dalam pengelolaan data kompleks antara Bank, Kementerian, SIPD, dan unit terkait lainnya, pendekatan RDBMS atau OODB sering mengalami kendala fleksibilitas dan performa, terutama saat menangani data bersarang dan relasi dinamis.

RDBMS membutuhkan banyak join untuk mengakses data yang saling terhubung, sementara OODB sulit diintegrasikan dalam sistem skala besar. Untuk itu, digunakan pendekatan DocumentDB (MongoDB) yang menyimpan data dalam format dokumen JSON dan memungkinkan struktur embedded antar entitas yang berkaitan.

Tujuan penggunaan DocumentDB:

- Menyatukan data terkait dalam satu dokumen agar lebih mudah diakses.
- Menghindari join yang kompleks dan mempercepat query.
- Mempermudah integrasi data dari sumber seperti XML, CSV, dan JSON.
- Mendukung penyimpanan data semi-terstruktur dengan fleksibel.
- Memungkinkan visualisasi dan analisis data secara real-time.

Dengan MongoDB, struktur data menjadi lebih efisien, mudah dikelola, dan mendukung kebutuhan sistem yang terus berkembang.

RDBMS/OODB TO SEMI-STRUCTURED SOLUTIONS

GRAPHDB (NEO4J)

RDBMS to Semi-Structured: GraphDB

Dalam pengelolaan sistem informasi yang kompleks seperti kolaborasi antara Bank dan Kementerian, pendekatan berbasis **RDBMS** (Relational Database Management System) dan **OODBMS** (Object-Oriented Database Management System) memiliki keterbatasan, khususnya ketika menangani relasi data yang bersifat dinamis, bertingkat, dan saling terhubung.

Untuk menjawab tantangan tersebut, digunakan pendekatan Graph Database (GraphDB) menggunakan Neo4j, yang memungkinkan setiap entitas seperti *Bank*, *Ministry*, *SIPD*, *Agency*, hingga *User* dimodelkan sebagai *simpul (nodes)* dan koneksi antar mereka sebagai *relasi (edges)*.

Tujuan dari pendekatan **GraphDB** ini adalah:

- Untuk menyederhanakan representasi hubungan antar entitas yang kompleks.
- Untuk mempermudah eksplorasi dan analisis pola keterkaitan antar institusi.
- Untuk menemukan pola relasi yang tidak dapat terdeteksi secara eksplisit menggunakan SQL konvensional pada RDBMS.

Dengan GraphDB, hubungan antar lembaga, alur data, serta struktur akses dapat divisualisasikan dan dianalisis secara lebih efisien dan real-time.

METHOD & ARCHITECTURE DIAGRAM STAGING XML

Methods and Staging Architecture to Support Data Interoperability from RDBMS to XML.

1. Metode Staging yang Diimplementasikan

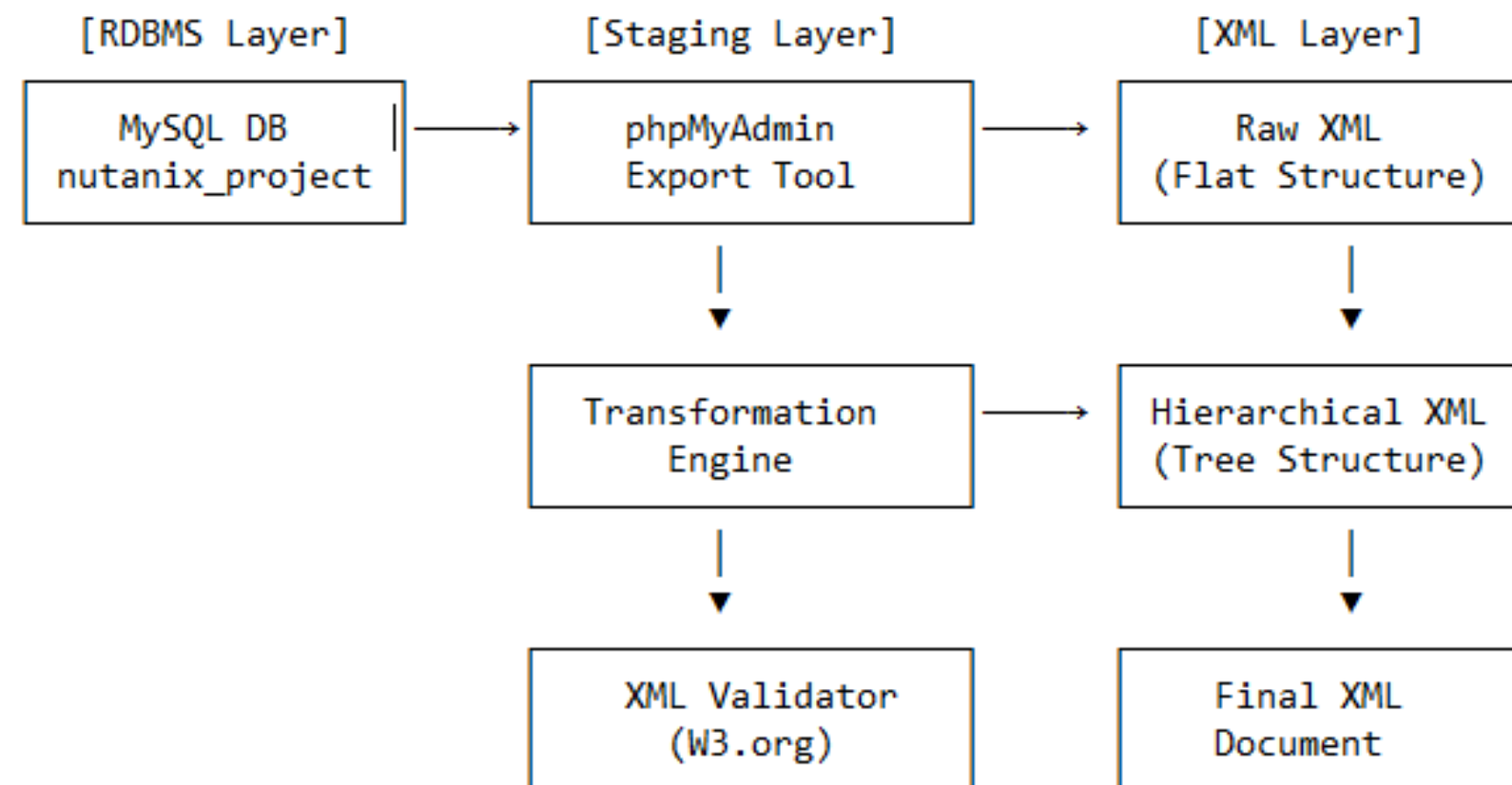
A. Proses Ekstraksi Data

- Sumber Data: Database MySQL 'nutanix_project'
- Tool: phpMyAdmin sebagai interface ekstraksi
- Format Output: XML sebagai format intermediate

B. Tahapan Transformasi

1. Export Raw Data: Database relational → XML flat structure
2. Transformasi Struktur: XML flat → XML hierarchical tree
3. Validasi: Menggunakan W3.org XML Validator

2. Arsitektur Staging



METHOD & ARCHITECTURE DIAGRAM STAGING XML

Methods and Staging Architecture to Support Data Interoperability from RDBMS to XML.

3. Struktur Data yang Ditransformasi

A. Entitas Utama

- Ministry: Kementerian Dalam Negeri RI
- Agency: Direktorat Jenderal Bina Administrasi Kewilayahan
- Bank: RBL Bank (Digital Bank dari India)
- SIPD: Sistem Informasi Perencanaan Pembangunan Daerah

B. Relasi Hierarkis dalam XML

```
ministries/  
├── ministry/  
├── agencies/  
│   ├── agency/  
│   └── data/  
├── sipds/  
│   ├── sipd/  
│   └── dataaccesses/  
├── costsavings/  
├── databasesystems/  
├── infrastructures/  
└── operationalimpacts/
```

4. Keunggulan Metode Staging

A. Interoperabilitas

- Format Standar: XML sebagai format universal
- Cross-Platform: Dapat dibaca berbagai sistem
- Struktur Hierarkis: Mempertahankan relasi antar entitas

B. Efisiensi Proses

- Automated Export: Menggunakan phpMyAdmin
- Structured Transform: Dari flat ke hierarchical
- Validation: Memastikan XML well-formed

5. Implementasi Teknis

A. Tools yang Digunakan

- 1.phpMyAdmin: Database export tool
- 2.Text Editor: Notepad++/VS Code untuk transformasi
- 3.W3.org Validator: XML validation

B. Format Output

- Raw XML: Struktur tabel relasional dalam XML
- Tree XML: Struktur hierarkis dengan parent-child relationships
- Validated XML: XML yang telah divalidasi struktur dan sintaksnya

METHOD & ARCHITECTURE DIAGRAM STAGING XML

Methods and Staging Architecture to Support Data Interoperability from RDBMS to XML

6. Studi Kasus: Nutanix Enterprise Cloud

A. Konteks Implementasi

- Organisasi: Kementerian Dalam Negeri RI
- Solusi: Nutanix Enterprise Cloud
- Tahun: 2024

B. Hasil Transformasi

- Cost Savings: Hardware reduction 40%, Storage reduction 90%
- Operational Impact: Response time 2 detik
- Storage Efficiency: Pengurangan hingga 90TB

Metode staging yang diimplementasikan berhasil mentransformasi data relasional menjadi format XML hierarkis yang mendukung interoperabilitas data. Pendekatan ini memungkinkan integrasi yang seamless antara sistem RDBMS tradisional dengan aplikasi modern yang menggunakan XML sebagai format pertukaran data.

7. Validasi dan Quality Assurance

A. Validasi Struktur

- XML Schema validation
- Well-formedness check
- Data integrity verification

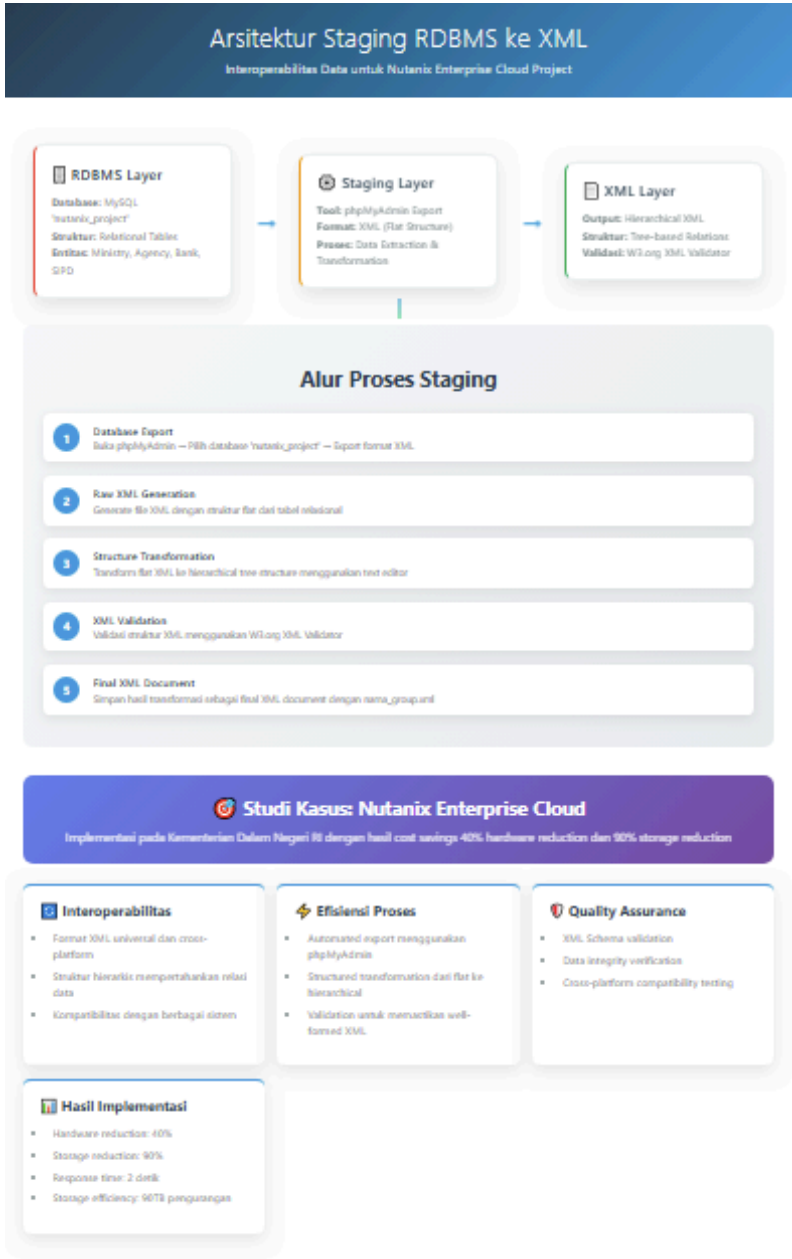
B. Interoperabilitas Testing

- Cross-platform compatibility
- Data parsing verification
- Integration testing

METHOD & ARCHITECTURE DIAGRAM STAGING XML

Visualization for Methods and Staging Architecture to Support Data Interoperability from RDBMS to XML

<https://symphonious-lamington-b3f215.netlify.app/>



METHOD & ARCHITECTURE DIAGRAM STAGING DOCDB

Methods and Staging Architecture to Support Data Interoperability from RDBMS/OODBMS to DocDB

1. Metode Staging yang Diimplementasikan

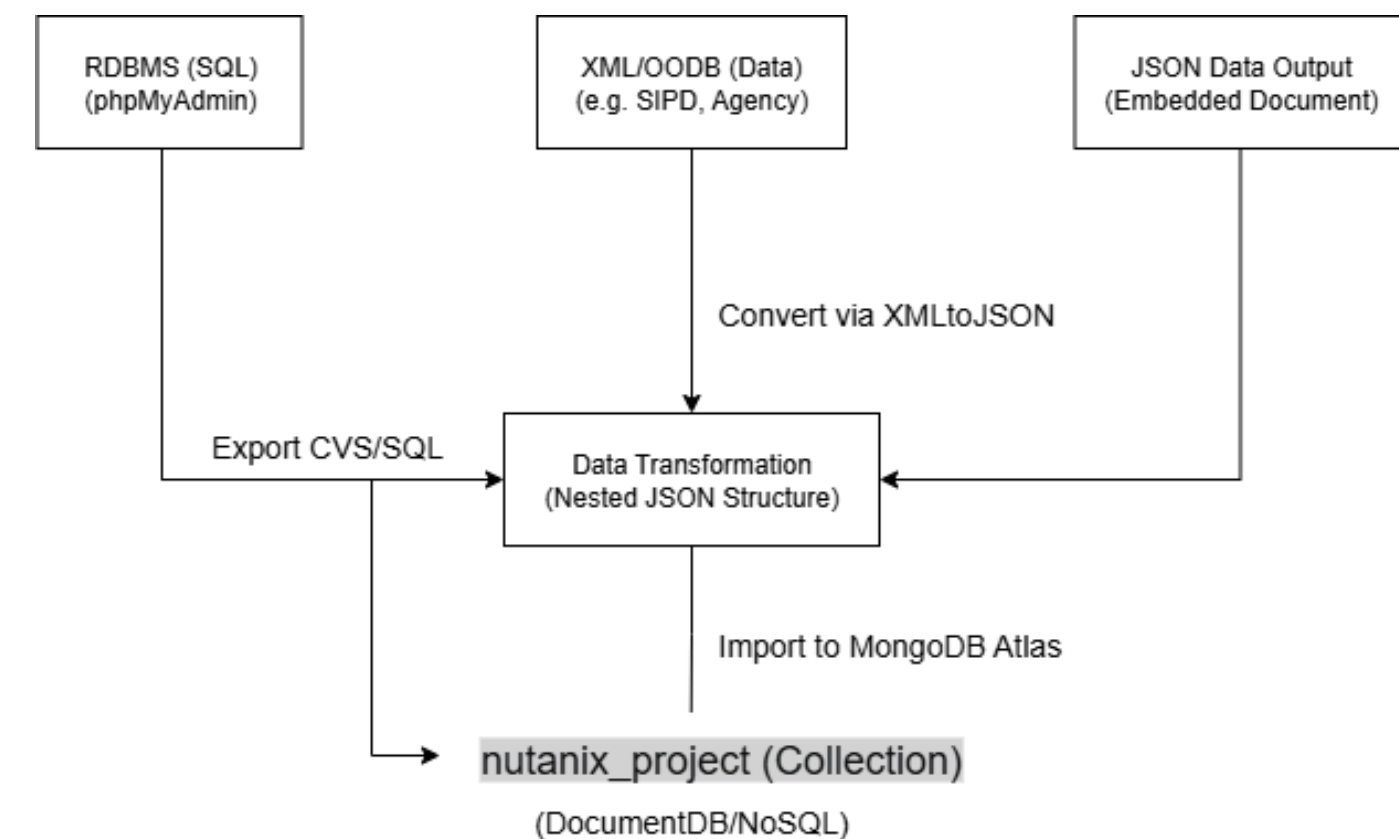
A. Proses Ekstraksi Data

- Sumber Data: Database RDBMS MySQL nutanix_project
- Tool: phpMyAdmin sebagai antarmuka basis data
- Format Output: XML dan CSV sebagai format antara

B. Tahapan Transformasi

- Export Raw Data: Data diekspor dari phpMyAdmin ke dalam format XML dan CSV
- Transformasi Struktur: XML diubah dari struktur flat menjadi struktur hierarkis tree
- Konversi ke JSON: Menggunakan tools online seperti xmltojson.com dan csvtojson.com
- Validasi: Struktur JSON diperiksa melalui MongoDB Compass dan Validator JSON

2. Arsitektur Staging



METHOD & ARCHITECTURE DIAGRAM STAGING DOCDB

Methods and Staging Architecture to Support Data Interoperability from RDBMS/OODBMS to DocDB

3. Struktur Data yang Ditransformasikan

A. Entitas Utama

- Bank: RBL Bank (Digital Bank dari India)
- Ministry: Kementerian Dalam Negeri RI
- Agency: Direktorat Jenderal Bina Administrasi Kewilayahan
- SIPD: Sistem Informasi Perencanaan Pembangunan Daerah
- Solution, CostSaving, DatabaseSystem, Infrastructure, Impact, DataAccess

B. Relasi Hierarkis dalam JSON (Embedded Model)

```
bank
├── ministry
│   ├── agency
│   │   └── data
│   ├── sipd
│   │   └── dataaccess
│   ├── costsaving
│   ├── databasesystem
│   ├── infrastructure
│   └── operationalimpact
└── solution
```

4. Keunggulan Metode Staging

A. Interoperabilitas

- Format Standar: XML dan JSON dapat digunakan lintas sistem
- Cross-Platform: Kompatibel dengan berbagai DBMS dan tools integrasi
- Embedded Structure: Menyederhanakan query dan mempercepat proses pencarian data

B. Efisiensi Proses

- Automated Export: Proses ekspor data cepat dari phpMyAdmin
- Transformasi Struktur: Data disusun sesuai relasi logis antar entitas
- Validasi Otomatis: Menggunakan Compass & JSON Linter

METHOD & ARCHITECTURE DIAGRAM STAGING DOCDB

Methods and Staging Architecture to Support Data Interoperability from
RDBMS/OODBMS to DocDB

5. Implementasi Teknis

A. Tools yang Digunakan

- phpMyAdmin: Untuk ekspor data RDBMS
- Text Editor: VS Code, Notepad++ untuk pengeditan data
- Online Converter: xmltojson.com, convertcsv.com
- MongoDB Atlas: Cloud DB Hosting
- MongoDB Compass: GUI MongoDB untuk validasi struktur
- MongoDB Charts: Dashboard interaktif untuk analisis data

B. Format Output

- Embedded JSON: Dokumen JSON bersarang dengan entitas utama & subdokumen
- Collection Tunggal: nutanix_project
- Struktur Konsisten: Menyesuaikan kebutuhan query dan agregasi MongoDB

6. Studi Kasus: Nutanix Enterprise Cloud

A. Konteks Implementasi

- Institusi: Kementerian Dalam Negeri Republik Indonesia
- Bank Partner: RBL Bank, India
- Solusi: Implementasi Nutanix Enterprise Cloud
- Tahun: 2024

B. Hasil Transformasi

- Storage Efficiency: Storage reduction sebesar 90TB
- Cost Saving: 40% hardware, 90% storage
- Operational Impact: Response time hanya 2 detik

METHOD & ARCHITECTURE DIAGRAM STAGING DOCDB

Methods and Staging Architecture to Support Data Interoperability
from RDBMS/OODBMS to DocDB

7. Studi Kasus: Nutanix Enterprise Cloud

A. Konteks Implementasi

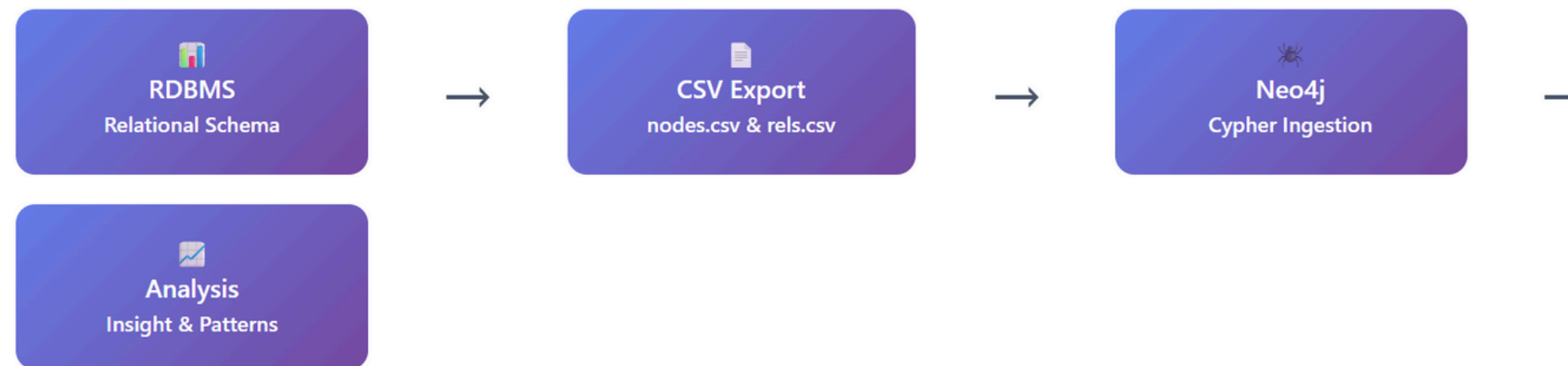
- Institusi: Kementerian Dalam Negeri Republik Indonesia
- Bank Partner: RBL Bank, India
- Solusi: Implementasi Nutanix Enterprise Cloud
- Tahun: 2024

B. Hasil Transformasi

- Storage Efficiency: Storage reduction sebesar 90TB
- Cost Saving: 40% hardware, 90% storage
- Operational Impact: Response time hanya 2 detik

METHOD & ARCHITECTURE DIAGRAM STAGING GRAPHDB

GraphDB: Staging Method & Architecture Diagram



Proses ini memanfaatkan **staging layer** berupa file CSV untuk menjembatani antara sistem relasional dan graph database. Dengan pendekatan ini, integrasi data ke Neo4j menjadi **modular, kontrolable**, dan siap untuk **eksplorasi relasi** menggunakan Cypher.

Staging adalah proses perantara yang digunakan untuk mentransformasikan data dari sistem **RDBMS ke GraphDB Neo4j**. Data yang semula tersimpan dalam bentuk tabel relasional diekspor ke dua file CSV, yaitu **nodes.csv** untuk menyimpan entitas dan **relationships.csv** untuk menyimpan hubungan antar entitas. File CSV ini berperan sebagai jembatan agar data dapat diimpor ke Neo4j menggunakan bahasa query **Cypher**. Melalui proses ini, data relasional dikonversi menjadi struktur graf yang lebih fleksibel dan mudah dianalisis.

METHOD & ARCHITECTURE DIAGRAM STAGING GRAPHDB

Diagram di samping menggambarkan alur lengkap bagaimana data dari sistem relasional (RDBMS) ditransformasikan ke dalam bentuk graf melalui metode staging. Proses diawali dari RDBMS, di mana data masih tersimpan dalam bentuk tabel dan saling terhubung menggunakan foreign key. Data ini tidak langsung dimasukkan ke Neo4j, melainkan terlebih dahulu diekspor ke dalam dua file CSV, yaitu nodes.csv untuk menyimpan data entitas seperti Bank, Ministry, atau SIPD, dan relationships.csv untuk menyimpan hubungan antar entitas seperti “**GOVERNS_AGENCY**” atau “**USES_DATABASE**”.

Kedua file CSV ini menjadi bagian dari lapisan staging, yaitu lapisan perantara yang memungkinkan data relasional dikonversi secara terstruktur ke format graf. Selanjutnya, data dari CSV tersebut diimpor ke dalam Neo4j menggunakan perintah **LOAD CSV** dengan bahasa **query Cypher**, yang secara otomatis membuat node dan relasi sesuai isi file.

Setelah semua data masuk ke Neo4j, maka terbentuklah struktur GraphDB, di mana setiap entitas menjadi node, dan setiap keterkaitan menjadi relationship. Pada tahap akhir, data ini bisa digunakan untuk analisis berbasis graf, seperti pencarian pola (pattern discovery), analisis jalur, deteksi anomali, dan lain-lain yang jauh lebih mudah dilakukan dibandingkan dengan sistem berbasis relasi konvensional.

Dengan staging seperti ini, proses migrasi data menjadi lebih fleksibel, terkontrol, dan siap untuk analisis lanjutan yang bersifat visual dan eksploratif.

QUERY RESULT ANALYSIS AND DISCUSSION XML

XPATH QUERIES

1. QUERY BERDASARKAN LOKASI NODES

Query untuk mendapatkan semua ministry nodes
`//database/ministries/ministry`

Query untuk mendapatkan semua agency dalam
ministry pertama
`//database/ministries/ministry[1]/agencies/agency`

Query untuk mendapatkan semua bank nodes
`//database/banks/bank`

Query untuk mendapatkan semua solution nodes di
dalam bank
`//database/banks/bank/solutions/solution`

Query untuk mendapatkan semua data nodes dalam
agency
`//database/ministries/ministry/agencies/agency/data`

Implementasi XPath untuk navigasi berdasarkan lokasi nodes menunjukkan kemampuan yang efektif dalam mengakses struktur hierarkis XML. Query seperti `//database/ministries/ministry` dan `//database/banks/bank` memungkinkan ekstraksi data secara langsung dari struktur dokumen yang kompleks. Pendekatan ini sangat berguna untuk aplikasi yang membutuhkan akses cepat ke elemen-elemen spesifik dalam dokumen XML besar, khususnya dalam konteks data pemerintahan dan perbankan yang memiliki struktur organisasi berlapis.

QUERY RESULT ANALYSIS AND DISCUSSION XML

XPATH QUERIES

2. QUERY BERDASARKAN LOKASI ATTRIBUTE

Query untuk mendapatkan ministry dengan id="min1"
`//database/ministries/ministry[@id="min1"]`

Query untuk mendapatkan bank dengan id="bank1"
`//database/banks/bank[@id="bank1"]`

Query untuk mendapatkan costsaving dengan
bank_id="1"
`//database/ministries/ministry/costsavings/costsaving[@bank_id="bank1"]`

Query untuk mendapatkan semua element yang
memiliki attribute bank_id
`//*[@bank_id]`

Query untuk mendapatkan solution dengan
ministry_id="min1"
`//database/banks/bank/solutions/solution[@ministry_id="min1"]`

Penggunaan attribute selector dalam XPath memberikan fleksibilitas tinggi dalam pencarian data spesifik. Query `//database/ministries/ministry[@id="min1"]` dan `//database/banks/bank[@id="bank1"]` mendemonstrasikan efisiensi dalam mengidentifikasi record berdasarkan identifier unik. Kemampuan ini sangat penting dalam sistem database yang memerlukan referensi silang antar entitas, seperti hubungan antara kementerian dan bank melalui attribute bank_id.

QUERY RESULT ANALYSIS AND DISCUSSION XML

XPATH QUERIES

3. QUERY BERDASARKAN TEKS

Query untuk mencari ministry dengan nama yang mengandung "Dalam Negeri"
`//database/ministries/ministry[contains(name, "Dalam Negeri")]`

Query untuk mencari bank dengan nama "RBL Bank"
`//database/banks/bank[name="RBL Bank"]`

Query untuk mencari data dengan tipe "Keuangan"
`//database/ministries/ministry/agencies/agency/data[data_type="Keuangan"]`

Query untuk mencari platform dengan teks "Cloud"
`//database/ministries/ministry/sipds/sipd[platform="Cloud"]`

Query untuk mencari country dengan teks "Indonesia"
`//database/ministries/ministry[country="Indonesia"]`

Fungsi text-based search menggunakan contains() dan equality operators menunjukkan kekuatan XPath dalam pencarian konten tekstual. Query seperti `//database/ministries/ministry[contains(name, "Dalam Negeri")]` memberikan kemampuan pencarian fuzzy yang berguna untuk aplikasi user interface. Pendekatan ini sangat efektif untuk sistem yang membutuhkan pencarian dinamis berdasarkan input pengguna.

QUERY RESULT ANALYSIS AND DISCUSSION XML

XPATH QUERIES

4. QUERY BERDASARKAN KONDISI (MENGUNAKAN OPERATOR)

Query untuk mendapatkan databasesystem dengan storage_capacity > 50
`//database/ministries/ministry/databasesystems/databasesystem[storage_capacity > 50]`

Query untuk mendapatkan costsaving dengan hardware_reduction >= 40
`//database/ministries/ministry/costsavings/costsaving[hardware_reduction >= 40]`

Query untuk mendapatkan impact dengan response_time < 5
`//database/ministries/ministry/operationalimpacts/impact[response_time < 5]`

Query untuk mendapatkan solution dengan implementation_year = 2024
`//database/banks/bank/solutions/solution[implementation_year = 2024]`

Query untuk mendapatkan ministry dengan id != "min2"
`//database/ministries/ministry[@id != "min2"]`

Implementasi conditional queries menggunakan operator matematis (>, >=, <, =, !=) menunjukkan kemampuan XPath dalam filtering data numerik. Query seperti `//database/ministries/ministry/databasesystems/databasesystem[storage_capacity > 50]` memungkinkan analisis data kuantitatif yang penting untuk decision making dalam manajemen infrastruktur teknologi.

QUERY RESULT ANALYSIS AND DISCUSSION

XML

XPATH QUERIES

5. QUERY BERDASARKAN POSISI DAN FUNGSI

Query untuk mendapatkan ministry pertama
`//database/ministries/ministry[1]`

Query untuk mendapatkan bank terakhir
`//database/banks/bank[last()]`

Query untuk mendapatkan jumlah ministry
`count(//database/ministries/ministry)`

Query untuk mendapatkan nama ministry yang dimulai dengan "Kementerian"
`//database/ministries/ministry[starts-with(name, "Kementerian")]`

Query untuk mendapatkan semua text nodes yang mengandung "Nutanix"
`//text()[contains(., "Nutanix")]`

Fungsi posisional dan built-in functions seperti `first()`, `last()`, `count()`, dan `starts-with()` memberikan kemampuan analisis yang sophisticated. Query `count(//database/ministries/ministry)` dan `//database/ministries/ministry[starts-with(name, "Kementerian")]` menunjukkan fleksibilitas dalam agregasi data dan pattern matching yang essential untuk reporting dan analytics.

QUERY RESULT ANALYSIS AND DISCUSSION

XML

XQUERY QUERIES

1. QUERY BERDASARKAN KONSTRUKSI (2 Queries)

[Query 1.1: Membuat struktur XML baru dengan informasi ministry]

```
for $ministry in //database/ministries/ministry
return
<ministry_info>
  <id>{data($ministry/@id)}</id>
  <name>{$ministry/name/text()}</name>
  <country>{$ministry/country/text()}</country>
  <agency_count>{count($ministry/agencies/agency)}</agency_count>
</ministry_info>
```

[Query 1.2: Konstruksi laporan bank dengan solusi]

```
for $bank in //database/banks/bank
return
<bank_report>
  <bank_name>{$bank/name/text()}</bank_name>
  <bank_type>{$bank/type/text()}</bank_type>
  <solutions>
    {
      for $solution in $bank/solutions/solution
      return <solution_name>{$solution/name/text()}</solution_name>
    }
  </solutions>
</bank_report>
```

XQuery menunjukkan keunggulan dalam transformasi dan konstruksi ulang struktur XML. Query konstruksi seperti pembuatan <ministry_info> dan <bank_report> memungkinkan reformatting data untuk berbagai keperluan presentasi dan integrasi sistem. Kemampuan ini sangat valuable dalam enterprise environment dimana data perlu ditransformasi untuk berbagai consumer applications.

QUERY RESULT ANALYSIS AND DISCUSSION

XML

XQUERY QUERIES

2. QUERY BERDASARKAN MANIPULASI: MERUBAH NILAI ATTRIBUTE (2 Queries)

[Query 2.1: Mengubah attribute id ministry menjadi "MOD_1"]

```
copy $doc := //database
modify (
  for $m in $doc/ministries/ministry[@id="min1"]
  return replace value of node $m/@id with "MOD_1"
)
return $doc
```

[Query 2.2: Mengubah attribute bank_id pada costsaving menjadi "BANK_NEW"]

```
copy $doc := //database
modify (
  for $c in
    $doc/ministries/ministry/costsavings/costsaving[@bank_id="bank1"]
  return replace value of node $c/@bank_id with "BANK_NEW"
)
return $doc
```

Fitur manipulasi menggunakan copy-modify-return pattern menunjukkan kekuatan XQuery dalam data modification. Query yang mengubah attribute id dari "min1" menjadi "MOD_1" dan bank_id menjadi "BANK_NEW" mendemonstrasikan kemampuan update data yang aman dan controlled. Pendekatan ini sangat penting untuk maintenance sistem dan data migration scenarios.

QUERY RESULT ANALYSIS AND DISCUSSION

XML

XQUERY QUERIES

3. QUERY BERDASARKAN MANIPULASI: MENAMBAH NODE BARU (2 Queries)

[Query 3.1: Menambah node baru ke dalam ministry]

```
copy $doc := //database
modify (
  for $m in $doc/ministries/ministry[@id="min1"]
  return insert node <established_year>1945</established_year> into $m
)
return $doc
```

[Query 3.2: Menambah bank baru ke dalam database]

```
copy $doc := //database
modify (
  insert node
    <bank id="bank2">
      <name>Bank Mandiri</name>
      <type>Commercial</type>
      <country>Indonesia</country>
    </bank>
  into $doc/banks
)
return $doc
```

Kemampuan insert node dalam XQuery memberikan fleksibilitas dalam extending struktur data existing. Query yang menambahkan <established_year>1945</established_year> dan bank baru menunjukkan bagaimana sistem dapat berkembang secara dinamis tanpa memerlukan redesign schema. Ini sangat berguna untuk aplikasi yang perlu adaptive terhadap changing business requirements.

QUERY RESULT ANALYSIS AND DISCUSSION

XML

XQUERY QUERIES

4. QUERY BERDASARKAN DEKLARASI NAMESPACE (2 Queries)

[Query 4.1: Menggunakan namespace untuk query dengan prefix]

```
declare namespace gov = "http://government.indonesia.org/schema";
for $ministry in //database/ministries/ministry
return
<gov:ministry_profile
xmlns:gov="http://government.indonesia.org/schema">
  <gov:name>{$ministry/name/text()}</gov:name>
  <gov:country>{$ministry/country/text()}</gov:country>
</gov:ministry_profile>
```

[Query 4.2: Namespace untuk financial data]

```
declare namespace fin = "http://finance.indonesia.org/schema";
declare namespace tech = "http://technology.indonesia.org/schema";

for $bank in //database/banks/bank
return
<fin:bank_profile xmlns:fin="http://finance.indonesia.org/schema"
  xmlns:tech="http://technology.indonesia.org/schema">
  <fin:name>{$bank/name/text()}</fin:name>
  <fin:type>{$bank/type/text()}</fin:type>
  <tech:solutions>
  {
    for $solution in $bank/solutions/solution
    return <tech:solution>{$solution/name/text()}</tech:solution>
  }
</tech:solutions>
</fin:bank_profile>
```

Implementasi namespace (declare namespace gov = "http://government.indonesia.org/schema") menunjukkan kemampuan XQuery dalam handling complex XML documents dengan multiple schemas. Pendekatan ini essential untuk enterprise integration dimana data dari berbagai source systems perlu diintegrasikan dengan maintaining their semantic context.

QUERY RESULT ANALYSIS AND DISCUSSION

XML

XQUERY QUERIES

5. QUERY BERDASARKAN KONDISI (FUNGSI LOGIKA STRING/NUMERIK) (2 Queries)

[Query 5.1: Kondisi menggunakan fungsi string dan numerik]

```
for $m in //database/ministries/ministry
let $len := string-length($m/name)
where $len > 30 and contains($m/name, "Indonesia")
return
<ministry_analysis>
  <name>{$m/name/text()}</name>
  <name_length>{$len}</name_length>
  <has_agencies>{if (count($m/agencies/agency) > 0) then "Yes" else "No"}</has_agencies>
</ministry_analysis>
```

[Query 5.2: Kondisi dengan fungsi numerik dan logika]

```
for $cs in //database/ministries/ministry/costsavings/costsaving
let $total := $cs/hardware_reduction + $cs/storage_reduction
where $cs/hardware_reduction >= 40 and $cs/storage_reduction >= 80
return
<savings_report>
  <bank_id>{data($cs/@bank_id)}</bank_id>
  <hardware_reduction>{$cs/hardware_reduction/text()}</hardware_reduction>
  <storage_reduction>{$cs/storage_reduction/text()}</storage_reduction>
  <total_reduction>{$total}</total_reduction>
  <efficiency_grade>{
    if ($total >= 120) then "Excellent"
    else if ($total >= 100) then "Good"
    else "Fair"
  }</efficiency_grade>
</savings_report>
```

XQuery mendemonstrasikan kemampuan advanced logic processing melalui penggunaan let, where, dan conditional expressions. Query yang menggunakan string-length(), contains(), dan mathematical operations menunjukkan kekuatan dalam complex data analysis. Kemampuan untuk membuat efficiency grading ("Excellent", "Good", "Fair") berdasarkan calculated values menunjukkan potensi untuk business intelligence applications.

QUERY RESULT ANALYSIS AND DISCUSSION

DOCDB

OPERATION QUERY MONGODB (Read One)

```
{ "BankID": 1 }
```

nutanix_project

+

cluster0.00pgpw.mongodb.net

>

nutanix_projectt

>

nutanix_project

Documents 11

Aggregations

Schema

Indexes 1

Validation

🕒

▼

{ "BankID": 1 }

➕ ADD DATA ▼

📄 EXPORT DATA ▼

✎ UPDATE

🗑 DELETE

💡

```

_id: ObjectId('68686ed3a2e09230441fb29d')
BankID: 1
Name: "RBL Bank"
Type: "Digital"
Country: "India"

```

```

_id: ObjectId('68686ed3a2e09230441fb29f')
InfrastructureID: 1
BankID: 1
MinistryID: 1
IT_Upgrades: "Implementasi Nutanix Enterprise Cloud"

```

```

_id: ObjectId('68686ed3a2e09230441fb2a1')
DatabaseID: 1
BankID: 1
MinistryID: 1
DatabaseType: "Oracle"
StorageCapacity: 100

```

```

_id: ObjectId('68686ed3a2e09230441fb2a2')
SolutionID: 1
BankID: 1
MinistryID: 1
SolutionName: "Implementasi Nutanix Enterprise Cloud"
ImplementationYear: 2024

```

nutanix_project

+

cluster0.00opgww.mongodb.net > nutanix_projectt > nutanix_project

Documents 11

Aggregations

Schema

Indexes 1

Validation

{ "BankID": 1 }

ADD DATA

EXPORT DATA

UPDATE

DELETE

```

databaseID : 1
BankID : 1
MinistryID : 1
DatabaseType : "Oracle"
StorageCapacity : 100

_id: ObjectId('68686ed3a2e09230441fb2a2')
SolutionID : 1
BankID : 1
MinistryID : 1
SolutionName : "Implementasi Nutanix Enterprise Cloud"
ImplementationYear : 2024

_id: ObjectId('68686ed3a2e09230441fb2a3')
SavingID : 1
BankID : 1
MinistryID : 1
HardwareReduction : 40
StorageReduction : 90

_id: ObjectId('68686ed3a2e09230441fb2a4')
ImpactID : 1
BankID : 1
MinistryID : 1
EfficiencyImprovement : "Pengurangan biaya penyimpanan hingga 90TB"
ResponseTime : 2

```

Analisis :

Query ini mengambil data bank dengan BankID = 1, yaitu "RBL Bank". Berguna untuk menampilkan detail entitas tertentu. MongoDB memudahkan akses data secara cepat tanpa perlu JOIN seperti di RDBMS.

QUERY RESULT ANALYSIS AND DISCUSSION DOCDB

OPERATION QUERY MONGODB (Read All)

{ }

nutanix_project	
cluster0.00pgpw.mongodb.net > nutanix_projectt > nutanix_project	
Documents 11	Aggregations Schema Indexes 1 Validation
{ }	
ADD DATA EXPORT DATA UPDATE DELETE	
<pre>{ "_id": "ObjectId('68686ed3a2e09230441fb29d')", "BankID": 1, "Name": "RBL Bank", "Type": "Digital", "Country": "India" }</pre>	
<pre>{ "_id": "ObjectId('68686ed3a2e09230441fb29e')", "MinistryID": 1, "Name": "Kementerian Dalam Negeri Republik Indonesia", "Country": "Indonesia" }</pre>	
<pre>{ "_id": "ObjectId('68686ed3a2e09230441fb29f')", "InfrastructureID": 1, "BankID": 1, "MinistryID": 1, "IT_Upgrades": "Implementasi Nutanix Enterprise Cloud" }</pre>	
<pre>{ "_id": "ObjectId('68686ed3a2e09230441fb2a0')", "AgencyID": 1, "MinistryID": 1, "Name": "Direktorat Jenderal Bina Administrasi Kewilayahan" }</pre>	
<pre>{ "_id": "ObjectId('68686ed3a2e09230441fb2a1')", "DatabaseID": 1, "BankID": 1 }</pre>	

nutanix_project	
cluster0.00pgpw.mongodb.net > nutanix_projectt > nutanix_project	
Documents 11	Aggregations Schema Indexes 1 Validation
{ }	
ADD DATA EXPORT DATA UPDATE DELETE	
<pre>{ "_id": "ObjectId('68686ed3a2e09230441fb2a2')", "ImpactID": 1, "BankID": 1, "MinistryID": 1, "EfficiencyImprovement": "Pengurangan biaya penyimpanan hingga 90TB", "ResponseTime": 2 }</pre>	
<pre>{ "_id": "ObjectId('68686ed3a2e09230441fb2a5')", "DataID": 1, "AgencyID": 1, "MinistryID": 1, "DataType": "Keuangan", "AccuracyLevel": "Akurat", "Timestamp": "2024-12-19T08:00:00" }</pre>	
<pre>{ "_id": "ObjectId('68686ed3a2e09230441fb2a6')", "SIPDID": 1, "MinistryID": 1, "DataIntegration": "Satu Data Indonesia", "Platform": "Cloud" }</pre>	
<pre>{ "_id": "ObjectId('68686ed3a2e09230441fb2a7')", "AccessID": 1, "SIPDID": 1, "UserID": 1, "Role": "Admin" }</pre>	

nutanix_project	
cluster0.00pgpw.mongodb.net > nutanix_projectt > nutanix_project	
Documents 11	Aggregations Schema Indexes 1 Validation
{ }	
ADD DATA EXPORT DATA UPDATE DELETE	
<pre>{ "BankID": 1, "MinistryID": 1, "SolutionName": "Implementasi Nutanix Enterprise Cloud", "ImplementationYear": 2024 }</pre>	
<pre>{ "_id": "ObjectId('68686ed3a2e09230441fb2a3')", "SavingID": 1, "BankID": 1, "MinistryID": 1, "HardwareReduction": 40, "StorageReduction": 90 }</pre>	
<pre>{ "_id": "ObjectId('68686ed3a2e09230441fb2a4')", "ImpactID": 1, "BankID": 1, "MinistryID": 1, "EfficiencyImprovement": "Pengurangan biaya penyimpanan hingga 90TB", "ResponseTime": 2 }</pre>	
<pre>{ "_id": "ObjectId('68686ed3a2e09230441fb2a5')", "DataID": 1, "AgencyID": 1, "MinistryID": 1, "DataType": "Keuangan", "AccuracyLevel": "Akurat", "Timestamp": "2024-12-19T08:00:00" }</pre>	

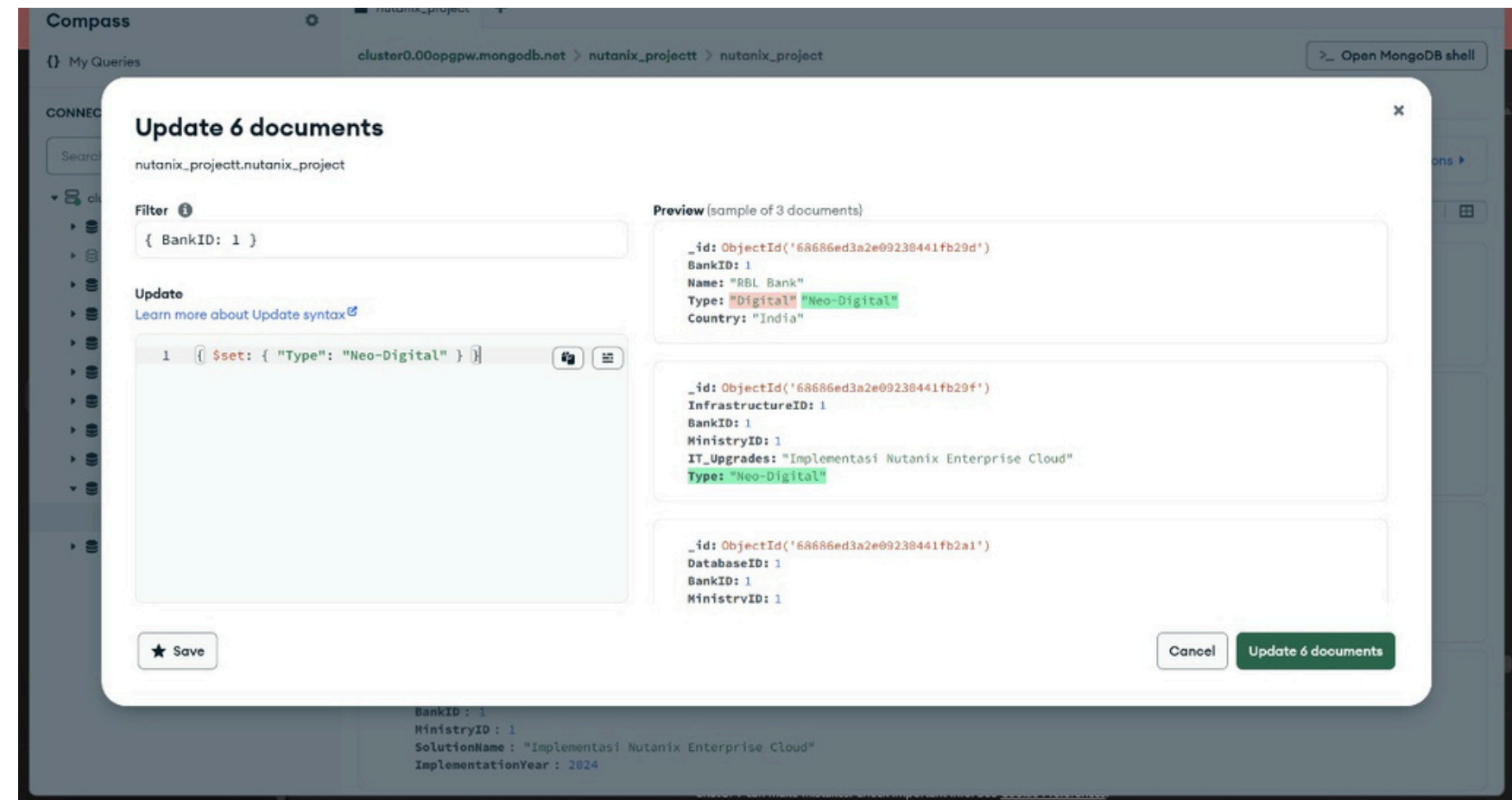
Analisis : Menampilkan seluruh dokumen di koleksi. Efisien untuk audit data, dan MongoDB mengembalikan hasil dalam array JSON tanpa batasan tabel seperti pada SQL.

QUERY RESULT ANALYSIS AND DISCUSSION DOCDB

OPERATION QUERY MONGODB (Update Query)

```
{ "BankID": 1 }  
{ $set: { "Type": "Neo-Digital" } }
```

Analisis: Mengubah tipe bank dari "Digital" menjadi "Neo-Digital". Dalam DocumentDB, perubahan dilakukan langsung ke dokumen tanpa constraint relasi seperti di RDBMS.

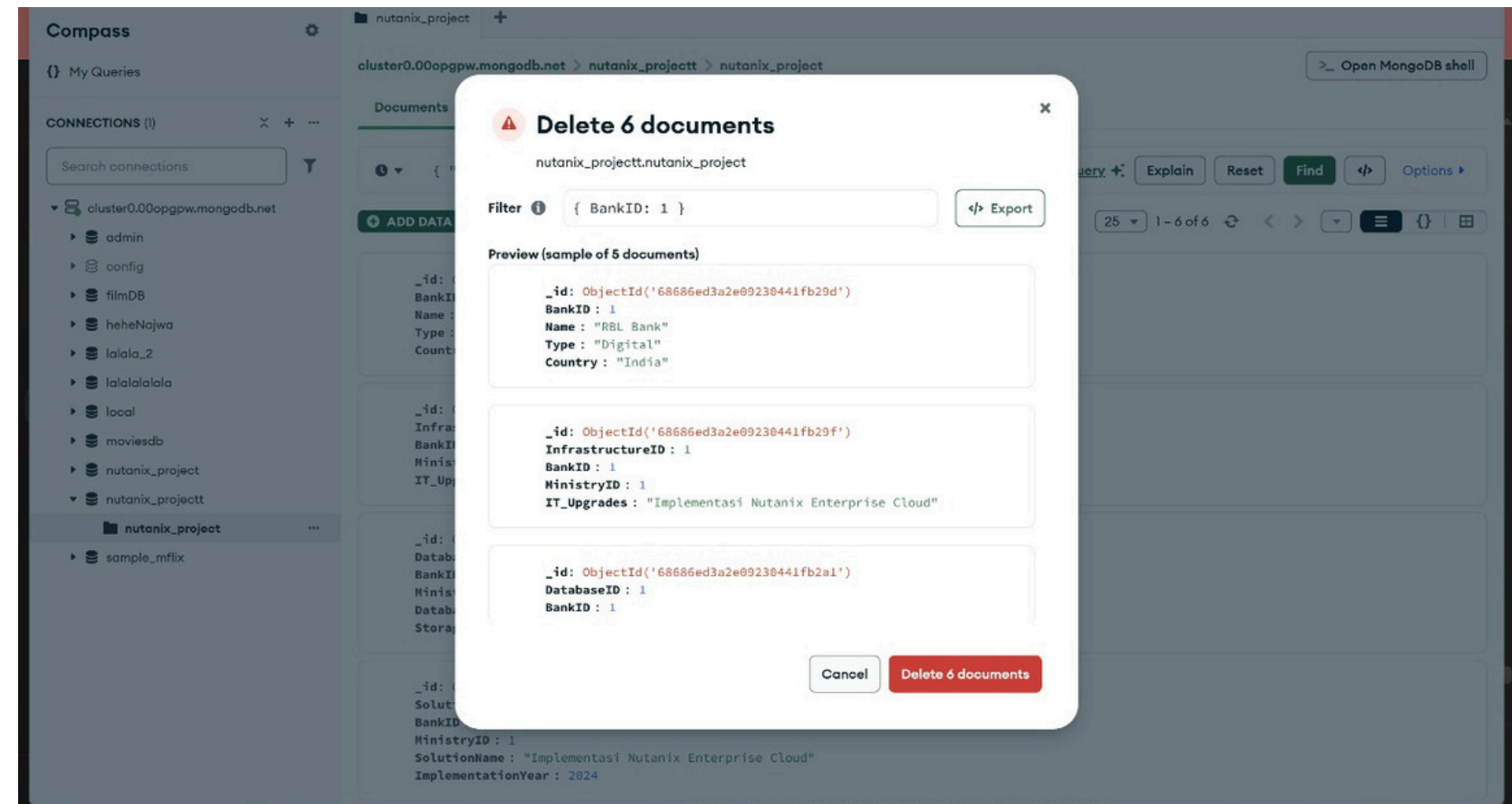


QUERY RESULT ANALYSIS AND DISCUSSION DOCDB

OPERATION QUERY MONGODB (Delete Query)

{ "BankID": 1 }

Analisis : Menghapus seluruh data bank dengan ID 1. Proses ini tidak memerlukan validasi foreign key seperti SQL. MongoDB memungkinkan penghapusan cepat satu entitas.



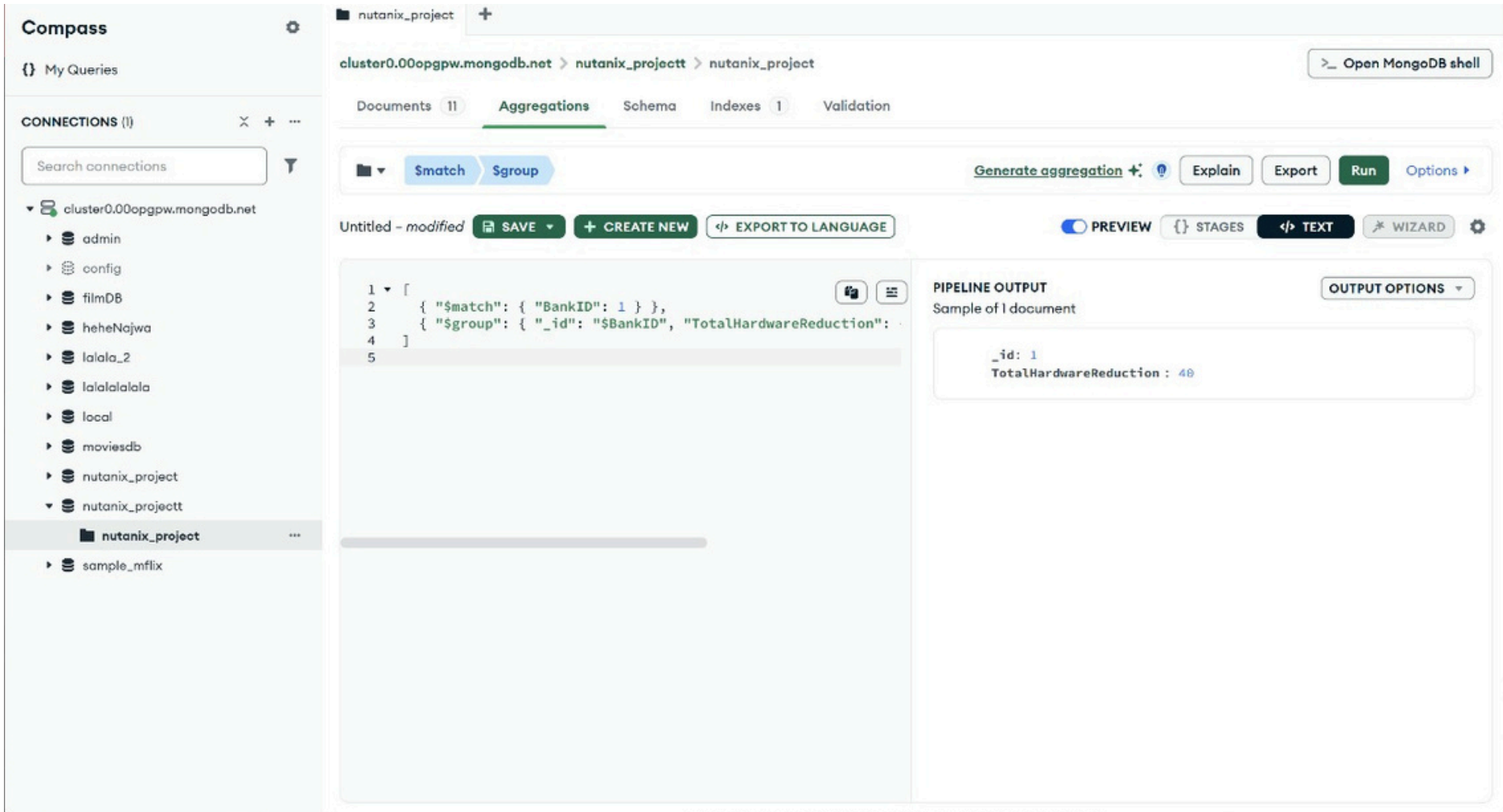
QUERY RESULT ANALYSIS AND DISCUSSION DOCDB

AGGREGATION QUERY MONGODB (\$match + \$group)

```
[ { "$match": { "BankID": 1 } }, { "$group": { "_id": "$BankID", "TotalHardwareReduction": { "$sum": "$HardwareReduction" } } } ]
```

Analisis:

Menghitung total pengurangan hardware untuk RBL Bank. Hasilnya menunjukkan penghematan sebesar 40 unit. Agregasi ini langsung mengakses field, tidak perlu join antar tabel.



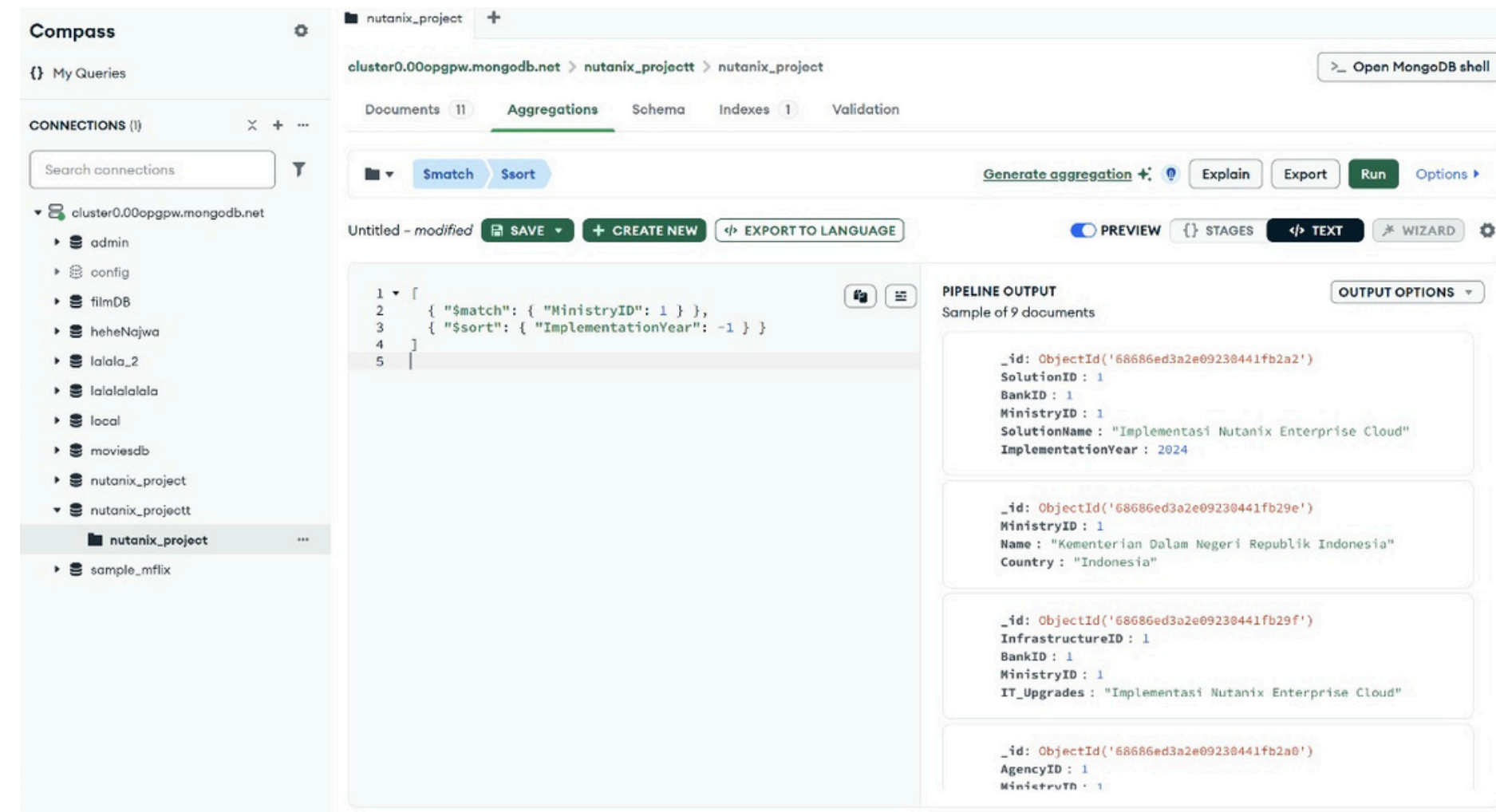
QUERY RESULT ANALYSIS AND DISCUSSION DOCDB

AGGREGATION QUERY MONGODB (\$match + \$sort)

```
[ { "$match": { "MinistryID": 1 } },  
  { "$sort": { "ImplementationYear": -1 } } ]
```

Analisis:

Menyortir solusi berdasarkan tahun implementasi terbaru. Dengan model dokumen, tidak perlu subquery atau join antar entitas karena data bersifat embedded.



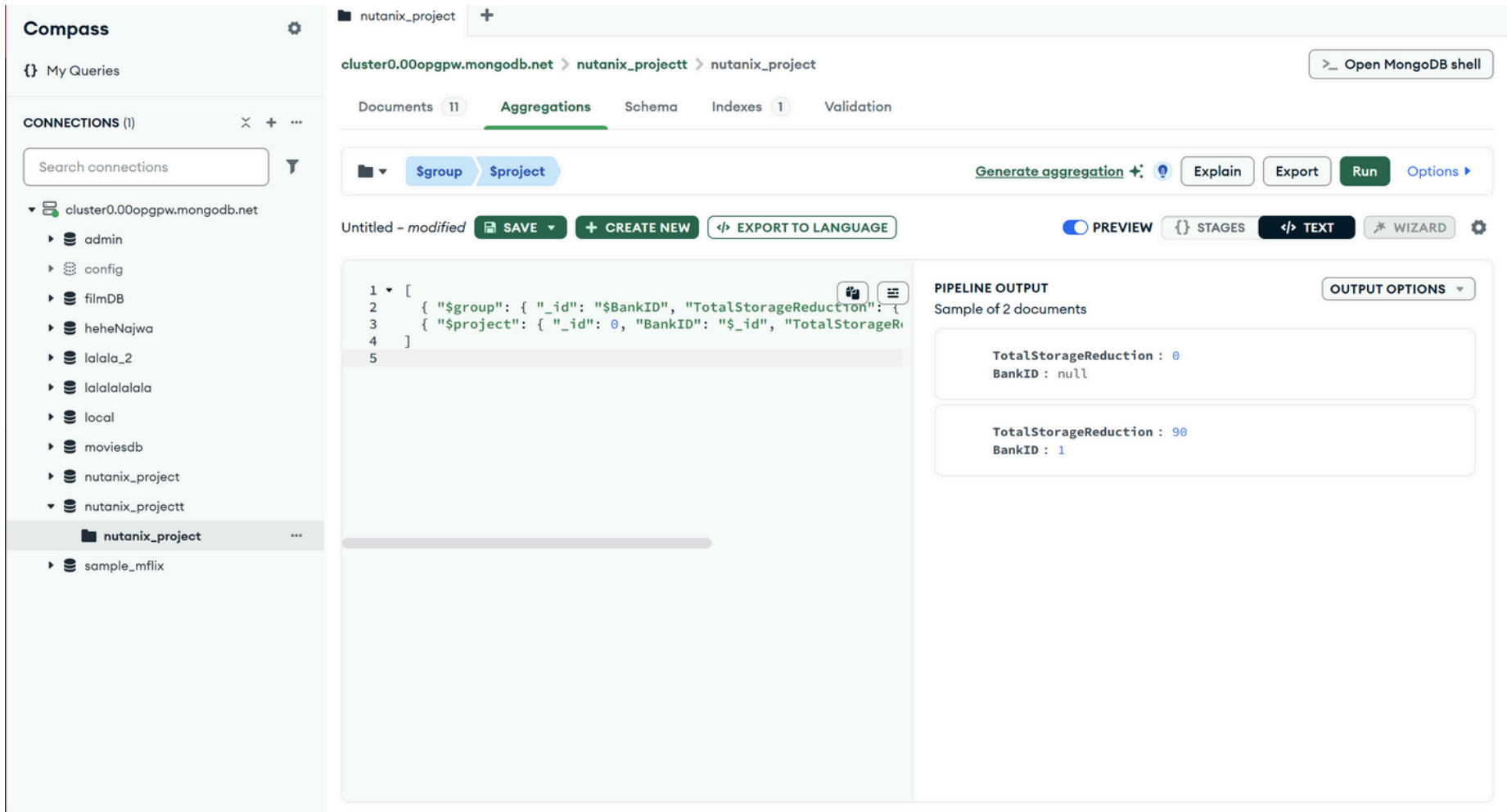
QUERY RESULT ANALYSIS AND DISCUSSION DOCDB

AGGREGATION QUERY MONGODB (\$group+ \$project)

```
[{ "$group": { "_id": "$BankID",  
  "TotalStorageReduction": { "$sum":  
    "$StorageReduction" } } }, { "$project": { "_id": 0,  
  "BankID": "$_id" "TotalStorageReduction": 1 } } ]
```

Analisis:

Menampilkan total storage saving per bank dalam bentuk ringkas. Berguna untuk insight penghematan penyimpanan, tanpa perlu normalisasi tabel seperti di RDBMS.



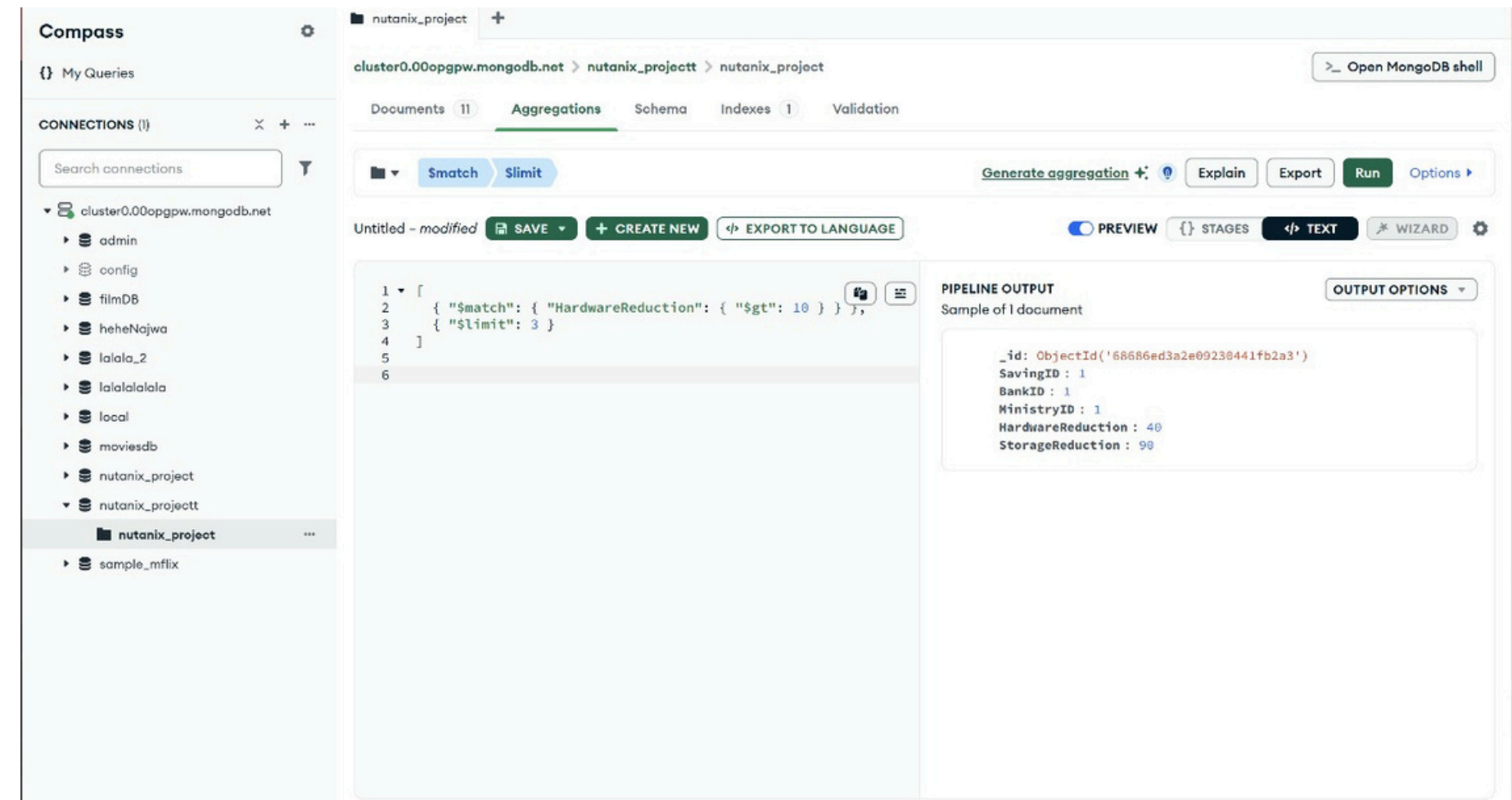
QUERY RESULT ANALYSIS AND DISCUSSION DOCDB

AGGREGATION QUERY MONGODB (\$group+ \$limit)

```
[{ "$match": { "HardwareReduction": { "$gt": 10 } }  
, { "$limit": 3 }]
```

Analisis:

Mengambil 3 data dengan penghematan hardware >10. Cocok untuk melihat performa solusi unggulan. Agregasi ini efisien karena filter dan limit bisa digabung dalam satu pipeline.



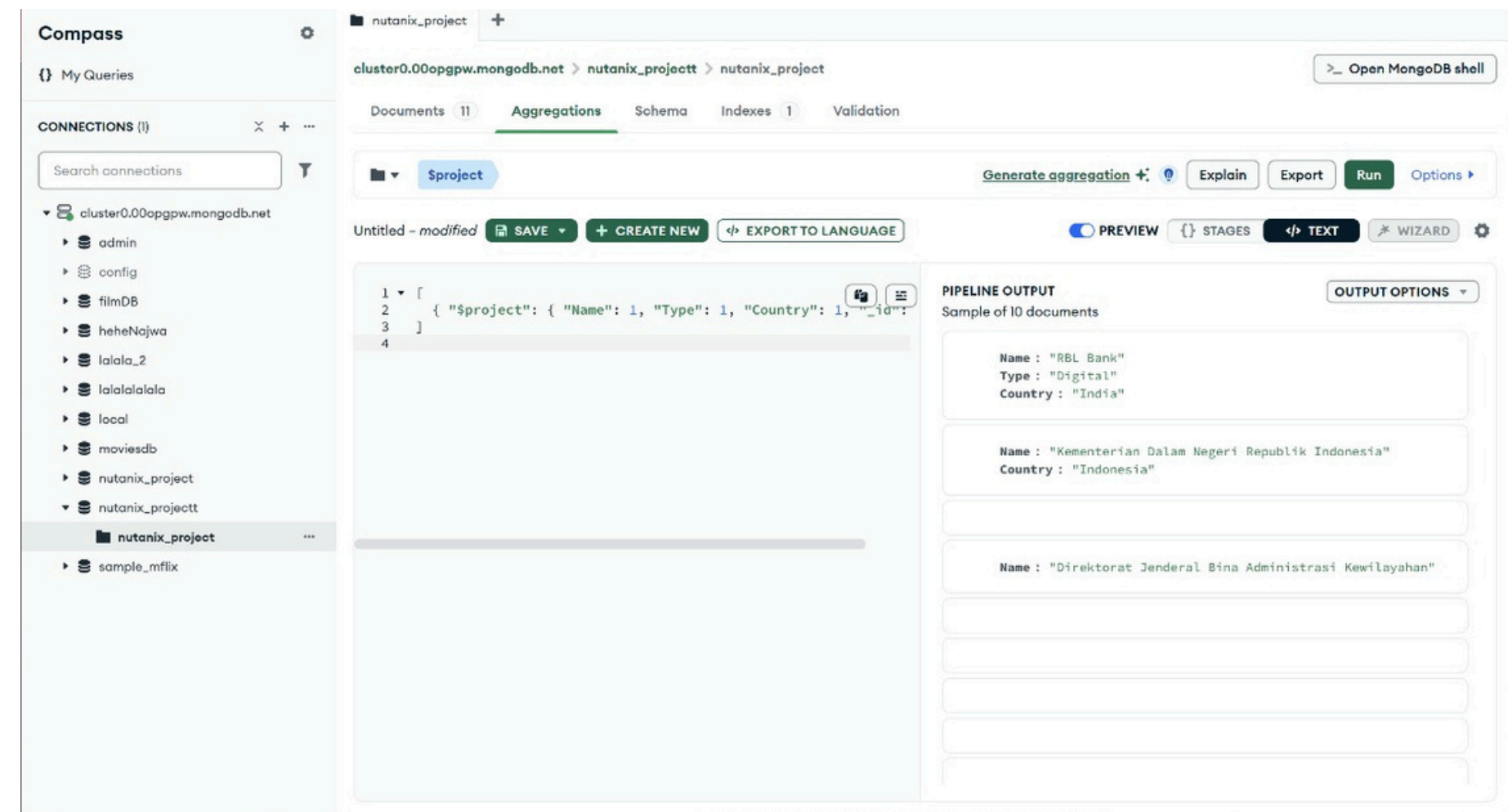
QUERY RESULT ANALYSIS AND DISCUSSION DOCDB

AGGREGATION QUERY MONGODB (\$project)

```
[{ "$project": { "Name": 1, "Type": 1, "Country": 1, "_id": 0 } }]
```

Analisis:

Menampilkan data bank secara ringkas tanpa field _id. Bermanfaat untuk membuat tampilan data di dashboard MongoDB Charts atau API front-end.

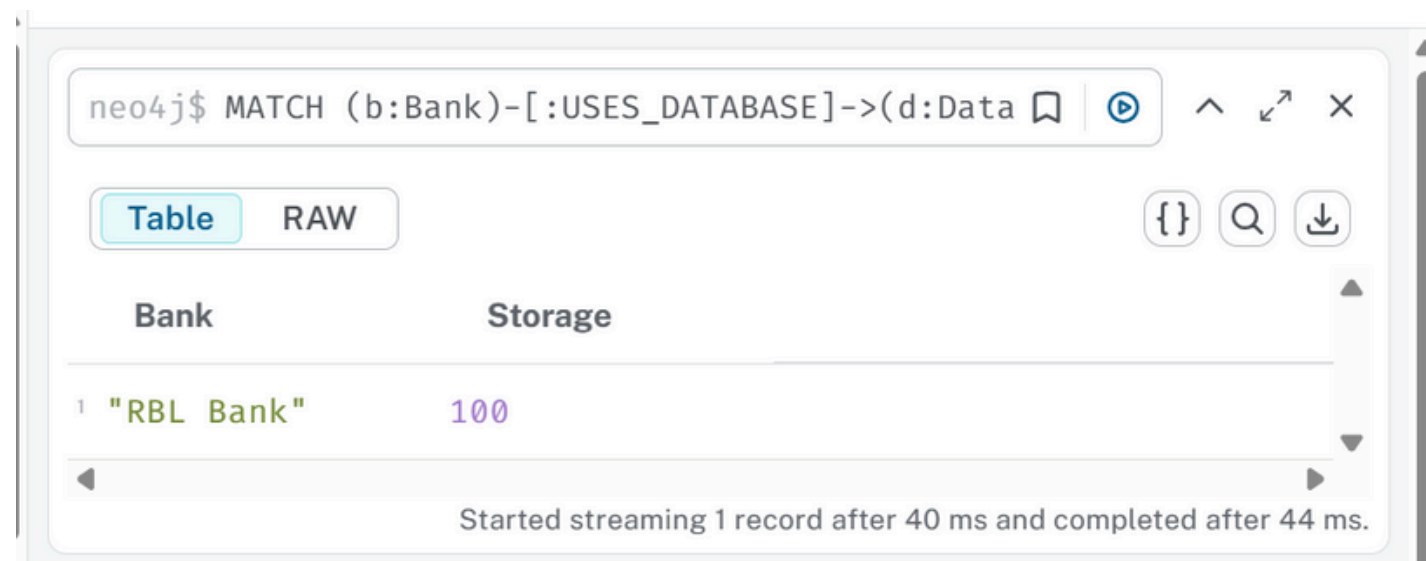


QUERY RESULT ANALYSIS AND DISCUSSION

GRAPHDB

Berikut adalah contoh pembahasan dari beberapa hasil query penting berdasarkan data yang telah di-transformasi dari RDBMS ke GraphDB:

ANALISIS QUERY #1 – Filtering & Sorting Database Oracle



	Bank	Storage
1	"RBL Bank"	100

Started streaming 1 record after 40 ms and completed after 44 ms.

Analisis:

Query ini bertujuan menampilkan nama bank yang menggunakan database Oracle, serta berapa besar kapasitas penyimpanannya. Dalam data kamu, RBL Bank menggunakan database **Oracle dengan kapasitas 100 TB**.

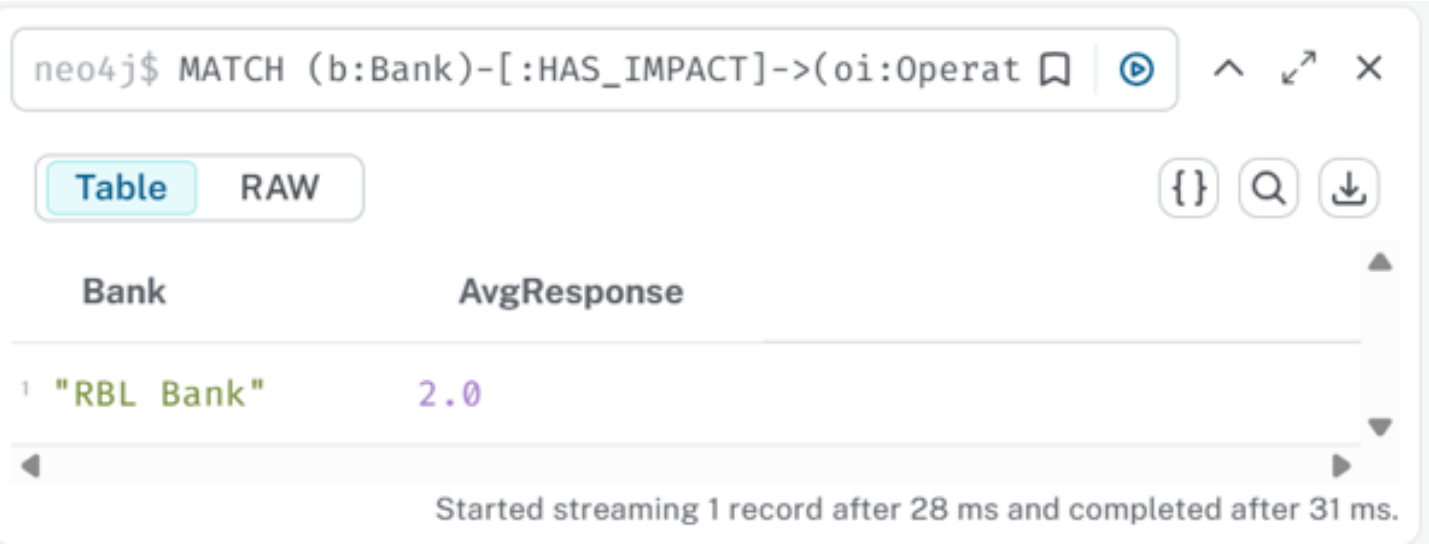
Dengan GraphDB, relasi seperti Bank → Database dapat dilacak langsung melalui edge **USES_DATABASE**, tanpa perlu melakukan JOIN antar-tabel seperti di RDBMS. Query juga lebih ringkas dan efisien.

QUERY RESULT ANALYSIS AND DISCUSSION

GRAPHDB

Berikut adalah contoh pembahasan dari beberapa hasil query penting berdasarkan data yang telah di-transformasi dari RDBMS ke GraphDB:

ANALISIS QUERY #2 – Agregasi Rata-Rata Waktu Respon



The screenshot shows the Neo4j Cypher query interface. The query entered is `neo4j$ MATCH (b:Bank)-[:HAS_IMPACT]->(oi:Operat`. The result is displayed in table view with two columns: **Bank** and **AvgResponse**. The first row shows **"RBL Bank"** with an **AvgResponse** of **2.0**. A status message at the bottom indicates: "Started streaming 1 record after 28 ms and completed after 31 ms."

Bank	AvgResponse
"RBL Bank"	2.0

Analisis:

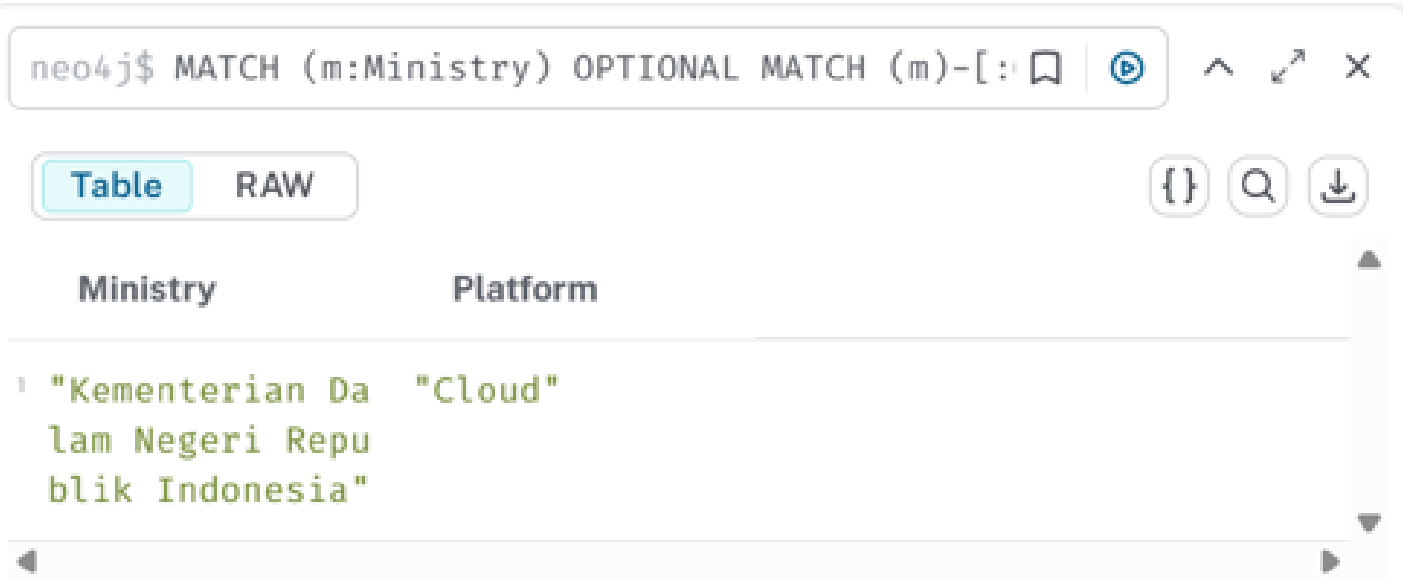
Query ini menghitung rata-rata waktu respon (dalam menit) yang dialami oleh bank dalam penerapan infrastruktur baru. Untuk RBL Bank, hasilnya adalah **2 menit sesuai data dari tabel OperationalImpact**. GraphDB mempermudah agregasi relasional seperti ini, karena data OperationalImpact terhubung langsung dengan Bank lewat relasi **HAS_IMPACT**. Ini tidak perlu subquery atau nested aggregation seperti di SQL RDBMS.

QUERY RESULT ANALYSIS AND DISCUSSION

GRAPHDB

Berikut adalah contoh pembahasan dari beberapa hasil query penting berdasarkan data yang telah di-transformasi dari RDBMS ke GraphDB:

ANALISIS QUERY #3 – OPTIONAL MATCH SIPD dari Ministry



The screenshot shows a Neo4j query interface. At the top, a text box contains the Cypher query: `neo4j$ MATCH (m:Ministry) OPTIONAL MATCH (m)-[: SIPD ](p:Platform)`. Below the query box, there are two tabs: "Table" (selected) and "RAW". To the right of the tabs are icons for JSON, search, and download. The results are displayed in a table with two columns: "Ministry" and "Platform". The first row of data shows "Kementerian Dalam Negeri Republik Indonesia" under the "Ministry" column and "Cloud" under the "Platform" column.

Ministry	Platform
"Kementerian Dalam Negeri Republik Indonesia"	"Cloud"

Analisis:

Query ini menggunakan *OPTIONAL MATCH* untuk memastikan semua Ministry tetap ditampilkan meskipun tidak punya relasi ke *SIPD*. Dari data, *Kementerian Dalam Negeri Republik Indonesia* memiliki *SIPD* dengan platform **Cloud**. Pendekatan ini sangat cocok dalam GraphDB karena fleksibel dan real-time, tanpa risiko kehilangan data seperti pada *LEFT JOIN* di SQL.

QUERY RESULT ANALYSIS AND DISCUSSION

GRAPHDB

Berikut adalah contoh pembahasan dari beberapa hasil query penting berdasarkan data yang telah di-transformasi dari RDBMS ke GraphDB:

ANALISIS QUERY #4 – Filtering Infrastructure Upgrade



The screenshot shows a Neo4j query interface. The query bar contains: `neo4j$ MATCH (b:Bank)-[:HAS_INFRASTRUCTURE]->(i`. Below the query bar, there are tabs for 'Table' and 'RAW', with 'Table' selected. To the right of the tabs are icons for JSON, Search, and Download. The results are displayed in a table with two columns: 'b.name' and 'i.it_upgrades'. The first row of data shows 'RBL Bank' and 'Implementasi Nutanix Enterprise Cloud'.

b.name	i.it_upgrades
"RBL Bank"	"Implementasi Nutanix Enterprise Cloud"

Analisis:

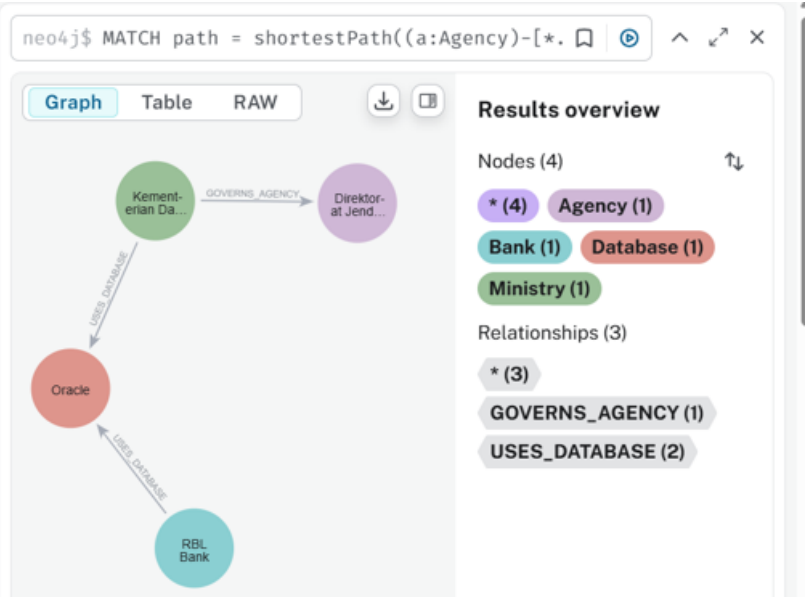
Query ini mencari bank yang melakukan peningkatan infrastruktur yang mengandung kata **"Nutanix"**. Hasilnya adalah *RBL Bank* dengan upgrade **"Implementasi Nutanix Enterprise Cloud"**. Fitur CONTAINS di GraphDB membantu pencarian teks dalam properti dengan mudah tanpa harus LIKE '%...%' seperti di RDBMS.

QUERY RESULT ANALYSIS AND DISCUSSION

GRAPHDB

Berikut adalah contoh pembahasan dari beberapa hasil query penting berdasarkan data yang telah di-transformasi dari RDBMS ke GraphDB:

ANALISIS QUERY #5 – Penemuan Pola Jalur Terpendek



Analisis:

Query ini digunakan untuk menemukan jalur terpendek antara **Agency dan Bank**, yang secara tidak langsung terhubung lewat node-node lain seperti Data, Ministry, dan SIPD.

Contohnya:

Agency → Data → Ministry → SIPD → Bank.

CONCLUSION

Dari keseluruhan tugas yang telah kami kerjakan, kami memahami bahwa setiap jenis basis data memiliki pendekatan dan struktur yang berbeda dalam menyimpan, mengelola, serta mengakses data. Pada bagian **XML (semi-structured data)**, kami belajar bagaimana mentransformasikan data dari model relasional ke dalam bentuk pohon (tree structure) yang fleksibel, serta menggunakan bahasa kueri seperti XPath dan XQuery untuk menavigasi dan memanipulasi data XML. Kami juga memahami pentingnya validasi dan interoperabilitas antar sistem melalui teknik staging, yang memungkinkan data dari RDBMS diubah menjadi format XML untuk digunakan di sistem lain.

Melalui bagian **DocumentDB (MongoDB)**, kami menyadari bahwa penyimpanan data dalam format dokumen JSON memberikan fleksibilitas tinggi, terutama dalam skema yang tidak tetap (schema-less). Kami belajar bagaimana data dapat di-query menggunakan perintah CRUD dan agregasi, serta bagaimana menghasilkan informasi visual melalui dashboard interaktif yang menyajikan insight dari data secara lebih intuitif. Proses ini mengajarkan kami tentang pentingnya konektivitas cloud, aksesibilitas data melalui federasi, dan bagaimana data dapat dikomunikasikan secara visual untuk mendukung pengambilan keputusan.

Sementara itu, dari eksplorasi **GraphDB (Neo4j)**, kami menangkap bahwa pendekatan graph sangat efektif untuk merepresentasikan hubungan antar entitas secara alami dan eksplisit. Kami belajar menyusun node dan relasi, memahami penggunaan Cypher query, dan mengenali keunggulan graph database dalam menemukan pola yang sulit diidentifikasi dengan pendekatan basis data konvensional, seperti shortest path atau pola relasi kompleks. Hal ini membuka pemahaman kami bahwa struktur graph sangat cocok digunakan untuk analisis jejaring, hubungan sosial, atau sistem yang saling terhubung.

Secara keseluruhan, kami memahami bahwa pemilihan jenis basis data sangat tergantung pada kebutuhan data dan tujuan penggunaannya. Tugas ini memperluas wawasan kami terhadap pendekatan modern dalam pengelolaan data dan memberikan pemahaman praktis terhadap interoperabilitas antar sistem, fleksibilitas struktur data, dan kekuatan eksplorasi pola yang hanya dapat dicapai melalui teknologi database tertentu.

REFERENCES

Overfishing removes species faster than they can reproduce, leading to collapsing fish stocks and ecosystem imbalances. Industrial practices, bycatch, and illegal fishing compound the problem. Establishing quotas, marine protected areas, and sustainable fishing methods are key solutions to restore fish populations and ensure future ocean food security.

ACKNOWLEDGE

Overfishing removes species faster than they can reproduce, leading to collapsing fish stocks and ecosystem imbalances. Industrial practices, bycatch, and illegal fishing compound the problem. Establishing quotas, marine protected areas, and sustainable fishing methods are key solutions to restore fish populations and ensure future ocean food security.



THANK YOU